

Field — Mac + iOS quick start

Set up Field on a new Mac and run the Capacitor app in the iOS Simulator (or on a physical iPhone).

iOS builds require macOS + Xcode. This cannot be done from Windows.

1. Install prerequisites

Tool	Notes
Xcode	App Store !' install !' open once and accept the license. Include iOS Simulator.
Xcode CLT	xcode-select --install if prompted
Homebrew	https://brew.sh
Node.js 22+	brew install node (or nvm / fnm)
CocoaPods	sudo gem install cocoapods then pod setup
Git	brew install git if needed
AWS CLI	brew install awscli — needed for RDS password + S3 credentials

Confirm:

```
node -v
npm -v
pod --version
xcodebuild -version
aws --version
```

2. Clone and install

```
git clone <repo-url> field
cd field
npm install
```

If ios/ is missing:

```
npx cap add ios
```

3. Configure .env

```
cp .env.example .env
```

1. Sign in to AWS (SSO / whatever this org uses), e.g. aws login or aws sso login.
2. Fetch the RDS password from Secrets Manager (command in .env.example).
3. Set DATABASE_URL in .env.
4. Leave AWS_REGION=us-west-1 and S3_BUCKET=field-dev-attachments as in the example.

Your Mac must be able to reach RDS field-dev (security group allows your public IP). Ask a teammate if the SG

needs updating.

Optional first-time schema (empty tables):

```
npm run db:schema
```

4. Run the local stack

```
npm run dev
```

- Web: <http://localhost:5173>
- API: <http://localhost:3000>

Keep this running while testing the mobile app.

5. iOS Simulator — day-to-day (live reload)

Preferred loop: Vite hot reload inside the native WebView.

Terminal 1 (already running):

```
npm run dev
```

Terminal 2 — point Capacitor at localhost Vite (do not use the default Android 10.0.2.2 URL):

```
CAP_SERVER_URL=http://127.0.0.1:5173 npm run cap:live
```

Then open Xcode and Run:

```
npx cap open ios
```

Or open `ios/App/App.xcworkspace` (use the workspace, not the `.xcodeproj`).

In Xcode:

1. Select an iPhone simulator.
2. Product !' Run (%⌘R).

UI edits in the React app should hot-reload in the simulator. API calls go through the Vite `/api` proxy to `:3000`.

6. iOS Simulator — bundled build

Production-style assets copied into the native project:

```
npm run cap:ios
```

That runs `build !' cap sync` (including `pod install`) !' opens Xcode. Pick a simulator !' Run.

Bundled iOS simulator API default is `http://127.0.0.1:3000` (see `src/api/client.ts`). Keep `npm run dev` (or at least the API) running on the Mac.

To leave live-reload mode and return to bundled assets, run `npm run cap:sync`.

7. Physical iPhone

1. Same Wi Fi as the Mac.
2. In Xcode: select your device, set a Team under Signing & Capabilities (Apple ID is fine for personal/dev).
3. Trust the developer on the phone if prompted.

Live reload:

```
npm run dev
npm run cap:live -- device
npx cap open ios
```

cap:live -- device uses the Mac's LAN IP for Vite. Run and select the phone in Xcode.

Bundled build (API must use the Mac's LAN IP, not localhost):

```
VITE_API_BASE=http://192.168.x.x:3000 npm run cap:sync
npx cap open ios
```

Replace 192.168.x.x with the Mac's IP (ipconfig getifaddr en0).

If attachment uploads fail after a LAN IP change:

```
npm run s3:cors
```

Useful commands

Command	Purpose
npm run dev	API + Vite
npm run dev:stop	Free ports 3000 + 5173
CAP_SERVER_URL=http://127.0.0.1:5173 npm run cap:live	Live reload !' Simulator
npm run cap:live -- device	Live reload !' physical device
npm run cap:sync	Build web + sync into ios/ / android/
npm run cap:ios	Sync + open Xcode
npm run s3:cors	Refresh S3 CORS for current LAN IP

Troubleshooting

Symptom	Fix
pod install fails	Install CocoaPods; from ios/App run pod install
Blank WebView / can't reach Vite	Simulator live reload must use CAP_SERVER_URL=http://127.0.0.1:5173, not 10.0.2.2
API errors on device	Set VITE_API_BASE=http://<mac-lan-ip>:3000 for bundled builds; phone and Mac on same Wi Fi

RDS connection refused	Re-login AWS; confirm SG allows this Mac's public IP; check DATABASE_URL
Signing error on device	Xcode !' Signing & Capabilities !' choose your Team
Stale live-reload URL	npm run cap:sync clears it and restores bundled dist/

App ID: app.field.mobile. QR activation is not implemented yet — the shell loads the same web UI as the browser.